WOOHOO V2 · IN-ROOM VOICE PLATFORM

Woohoo V2 Data Flow

What happens between a guest speaking and a light changing: every component the request passes through, every store it reads, and the four points where it can be stopped. Drawn from the running system, with the parts that are still only specified marked as such.

---

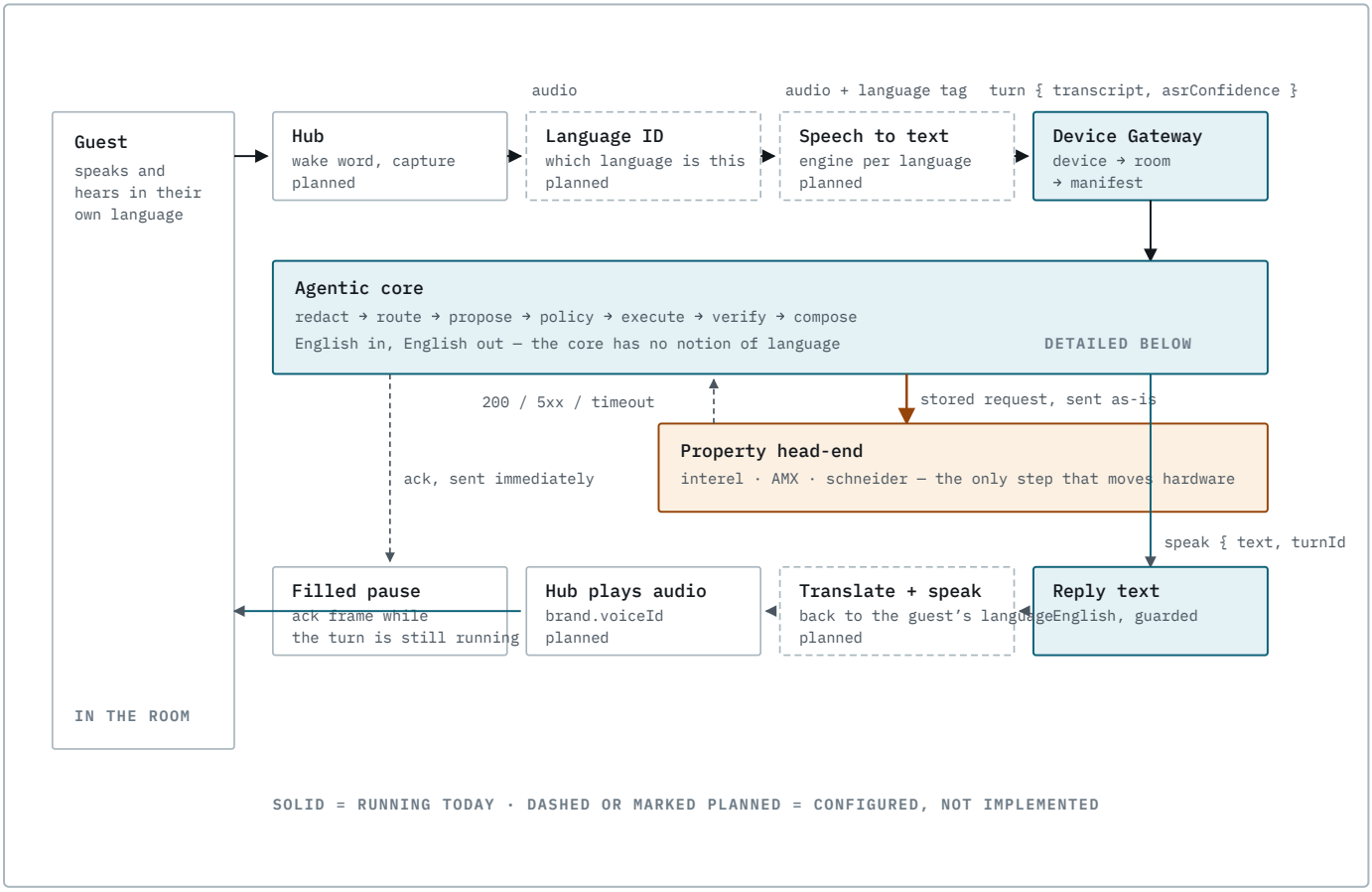
22 tools **6** risk tiers **4** routing lanes **9** guardrail controls **11** domain events **3** automation adapters

---

01 The full loop

A guest speaks in their own language and hears an answer in the same one. Between those two moments the request crosses from the room, to the cloud, out to the property's control hardware, and back.

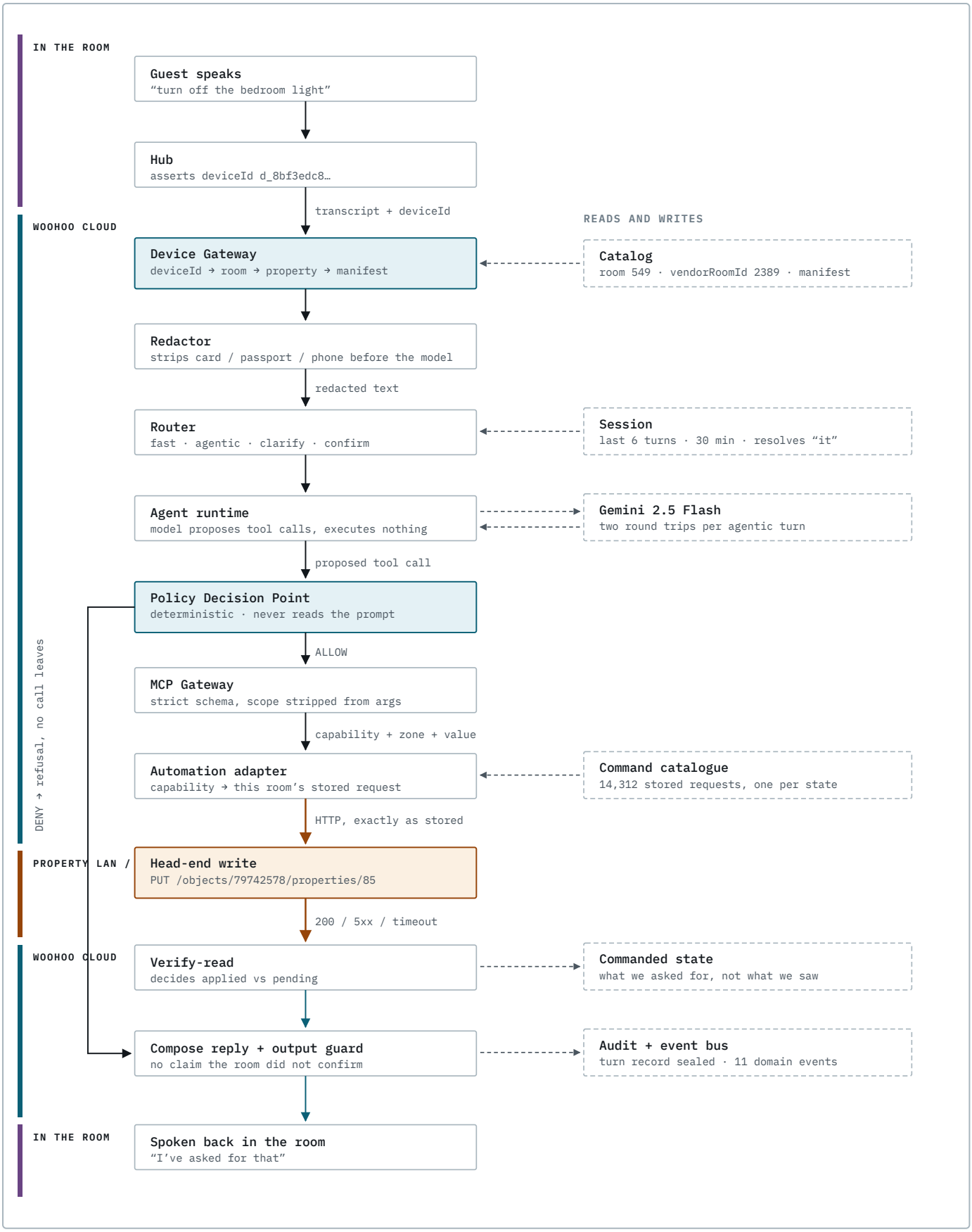
Boxes with a solid border are running today. Boxes marked **planned** are named in the property configuration — `wakeWordModel`, `asr.primary`, `languages`, `brand.voiceId` — but have no implementation behind them yet, so the voice path is currently entered as text.



The round trip. The agentic core is monolingual by design: language is resolved on the way in and restored on the way out, so nothing in the policy or automation path has to reason about it.

Inside the core

The same turn, expanded. The left rail marks which trust zone each step runs in — the request crosses that boundary four times, and every crossing is a place where identity, scope or reachability can fail. Only one step changes anything physical; everything above it exists to decide whether that step is allowed to happen.

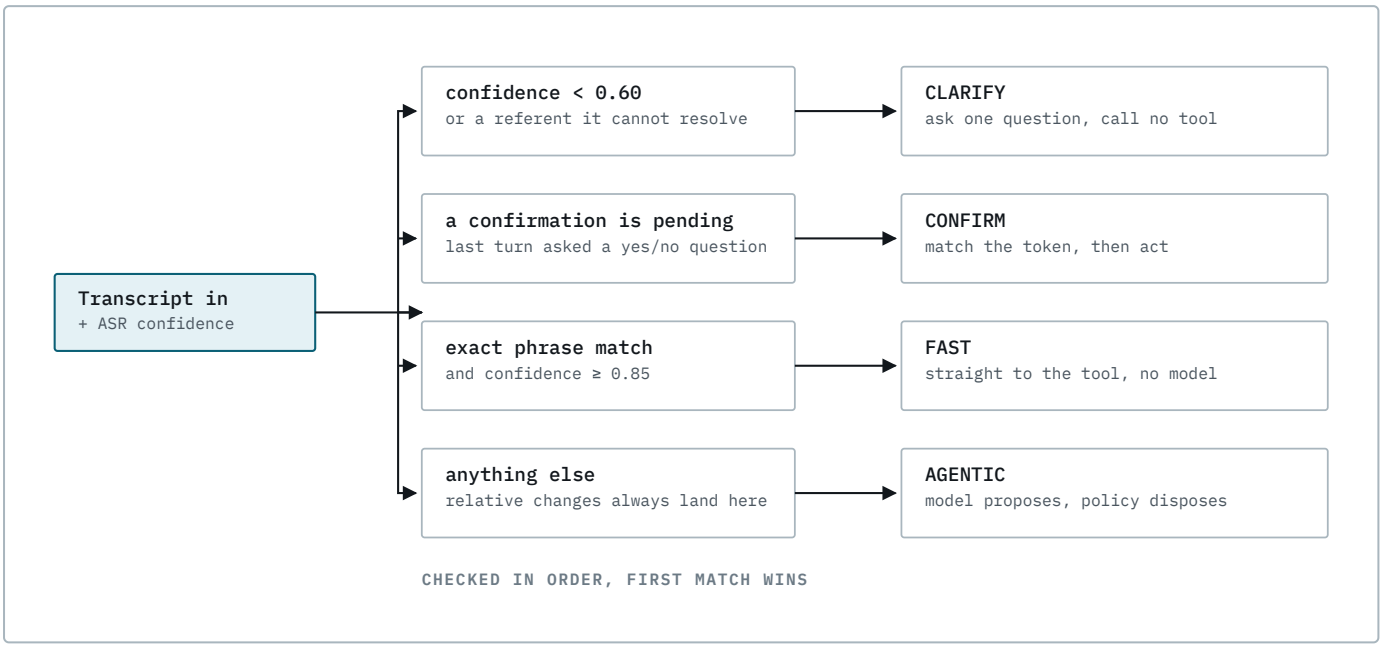


The turn pipeline. Solid arrows carry the request, dashed arrows are reads and writes against stores, amber is the only leg that reaches physical hardware, and the red return path is what happens when policy says no.


in the room
Woohoo cloud
property / vendor
touches hardware
blocked

02 Which lane a sentence takes

Not every utterance is worth a second of model latency, and not every utterance is safe to act on immediately. The router picks one of four lanes before the model is ever called.



Lane selection happens before any model call. The fast lane exists for latency and is kept deliberately narrow; anything carrying a pronoun or a relative change is excluded from it by rule.

> **Every lane converges on the same policy gate.** The fast lane skips the model, not the rules — a fast-lane action is still tier-checked, still scope-bound, still audited.

03 What the policy gate checks

Risk is a property of the tool, fixed at design time — not of the model's confidence and not of how the guest phrased it. The tier determines what the platform demands before the action reaches the world.

TIER	MEANING	REQUIRED BEFORE IT RUNS	TOOLS
T0	Read-only, no side effects	Scope check	6
T1	Reversible physical, low consequence	Scope, capability present, value in bounds	8
T2	Involves a human or an external commitment	Scope, duplicate check, rate limit	5
T3	Money, or hard to reverse	Spoken confirmation in a separate turn	1
T4	Safety-critical or staff authority	Agent may only raise it; a human acts	1
T5	Prohibited	No tool exists — absence is the control	0

> **There is no door-lock tool.** T5 is enforced by there being nothing to call, which is why a fully compromised model still cannot unlock a door. The same logic makes the room un-addressable: no tool anywhere takes a room parameter, so "unlock room 402" fails structurally rather than by refusal.

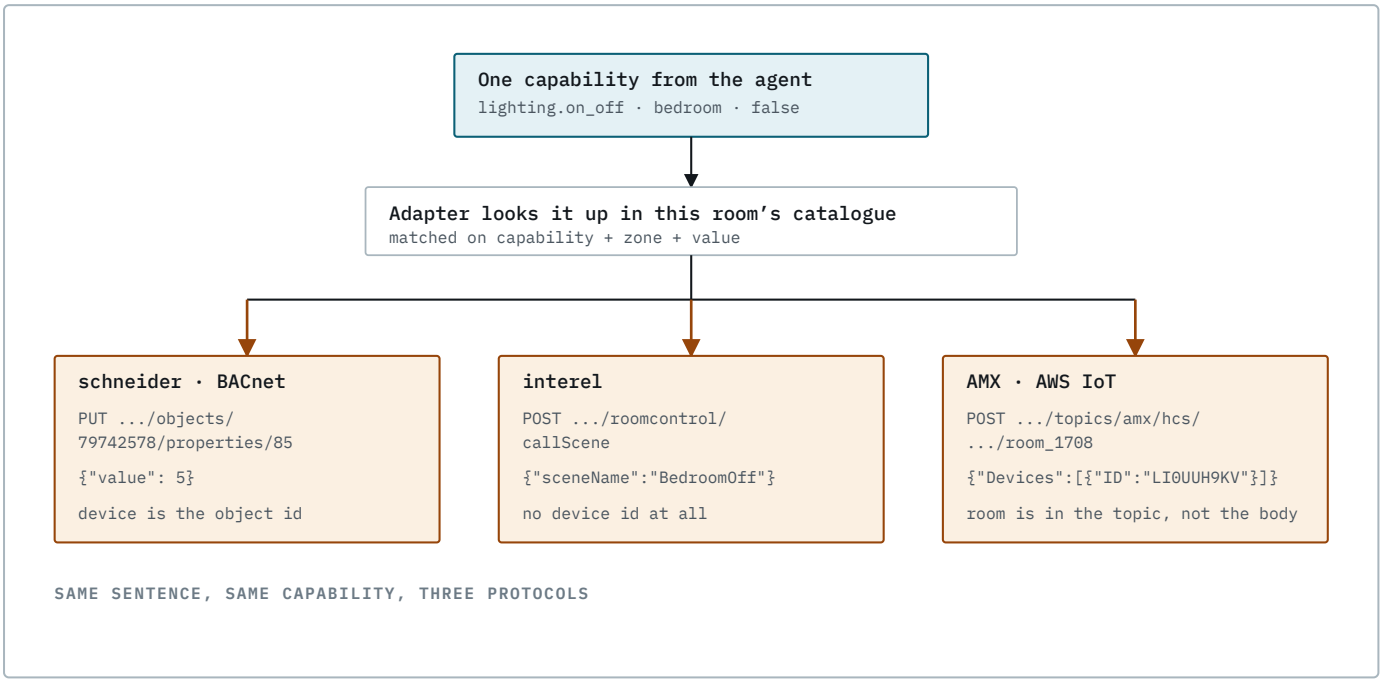
04 The tool surface

Twenty-two tools, and the model only ever sees the subset this room can actually do — the capability manifest filters the list before the prompt is built. A suite with no curtains is not offered `curtains.set`.

TOOL	TIER	NOTES
`room.get_state`	T0	Current room reading
`service.get_status`	T0	
`fnb.get_menu`	T0	
`fnb.get_order_status`	T0	
`stay.get_info`	T0	Checkout time, wifi, folio
`property.search_knowledge`	T0	Result is untrusted content
`lighting.set`	T1	verify-read
`climate.set_temperature`	T1	verify-read, bounded by property policy
`climate.set_mode`	T1	verify-read
`curtains.set`	T1	verify-read, degrades to open/close
`dnd.set`	T1	verify-read
`scene.apply`	T1	verify-read
`tv.control`	T1	
`media.control`	T1	
`preferences.set`	T1	Consent-gated
`service.create_request`	T2	Dispatches a human
`service.cancel_request`	T2	
`stay.request_late_checkout`	T2	External commitment
`stay.set_wakeup_call`	T2	
`staff.connect`	T2	
`fnb.place_order`	T3	Spends money — explicit confirmation required
`staff.escalate_urgent`	T4	Raises to a human; the agent cannot resolve it

05 Where the vendor vocabulary stops

This is the boundary that makes a new property configuration rather than code. Above the adapter, nothing knows the word “BACnet”. Below it, the same capability becomes three completely different requests — and for one vendor, the room is identified in the URL rather than the body.

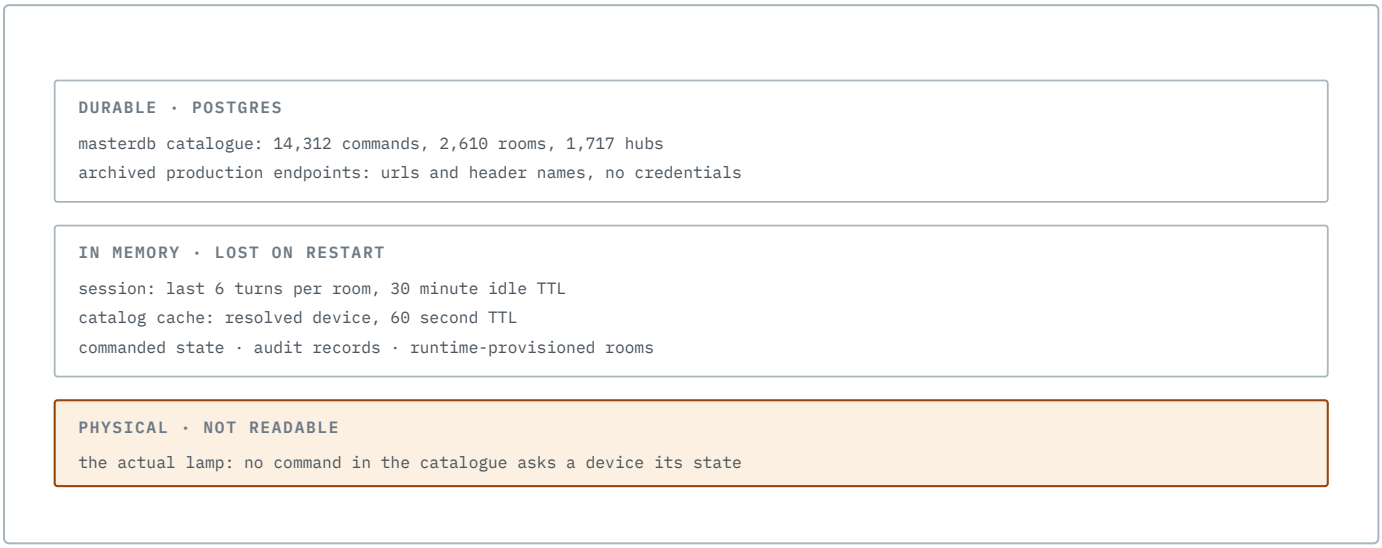


The adapter is the only component that reads the stored request. It composes nothing: the URL, method and body are copied from the row and sent as they are.

> **AMX puts room identity in the MQTT topic.** All 253 AMX rooms share the same device IDs and the same `ControllerID` in the payload — only the topic differs. Read the body alone and every room looks identical, which is a trap worth knowing before anyone debugs it.

06 Where state lives, and what survives

Four stores, three lifetimes. The distinction that matters operationally: everything in the middle band is in process memory, so a restart takes it with it — including rooms provisioned at runtime.



The bottom band is the constraint everything else bends around: with no read path, the platform can know what it asked for and never what is true.

> **This is why a successful write reports `pending`, never `applied`.** The assistant says “I’ve asked for that” rather than “done”, because claiming a light changed when nothing can confirm it is the failure that costs a guest’s trust in everything else the system says.

07 The guardrail chain

Nine controls across eight stages. Most of them succeed by producing nothing — a tool that was never offered, a field that was stripped, a room that could not be addressed — which is exactly what makes them hard to trust. Each turn records what every control did.

STAGE	CONTROL	WHAT IT DOES ON A TURN
gateway	Ambient scope	Tenant, property and room derived from the device record, never the sentence
identity	Scope stripping	Scope fields removed from tool arguments so the model cannot set them
redaction	PII redaction	Card, passport and phone patterns removed before the transcript reaches the model
tool-surface	Capability filtering	Only tools this room can perform are offered
tool-surface	Strict schema validation	Arguments rejected on shape, not coerced
policy	Policy Decision Point	Tier rules applied without reading the prompt or the reasoning
execution	Verify-read	Re-reads after a write; downgrades to pending when it cannot confirm
resilience	Degradation ladder	Levels 0–5; at 2 and above the agentic lane is bypassed entirely
output	Output guard	Blocks a reply that claims more than the tool results support

08 What the rest of the estate hears

Every turn seals an append-only record and emits domain events. These are the integration surface for staff apps, dashboards and anything downstream — consumers never read the orchestrator's internals.

EVENT	RAISED WHEN
`woohoo.turn.completed`	Every turn, including refusals
`woohoo.policy.denied`	A proposed tool call was blocked
`woohoo.escalation.raised`	T4 path — needs a human, with an acknowledgement deadline
`woohoo.service_request.created`	Housekeeping or maintenance dispatched
`woohoo.service_request.updated`	Status moved
`woohoo.fnb_order.placed`	Confirmed order, after the T3 confirmation turn
`woohoo.device_state.changed`	Room state moved
`woohoo.device.online`	Hub reconnected
`woohoo.device.offline`	Hub dropped — the room is deaf until it returns
`woohoo.stay.checked_out`	Triggers guest-data erasure
`woohoo.capability_manifest.updated`	A room type gained or lost a capability

09 Build status of every component

The boxes marked **planned** in the first diagram, spelled out. Each one is named in the property configuration, which is why it is drawn in the flow rather than left off — the shape is agreed, the implementation is not there. Worth settling before anyone plans against them.

`speech`	**Not built.** Transcripts arrive as text over HTTP or WebSocket. No wake word, no ASR, no TTS. The property config carries `asr.primary`, `languages` and `defaultLanguage`, so the shape is reserved — nothing fills it.
`language ID`	**Not built.** There is no language-detection step and no per-language routing. Replying in the guest's own language is a config field today, not behaviour.
`device auth`	**Asserted, not proven.** `deviceId` is a field in the request body. A `certificateSerial` exists on the device record and nothing verifies it, so anyone who can reach the port can claim any room.
`persistence`	**In memory only.** Sessions, audit records and runtime-provisioned rooms die on restart. A single instance means one reboot takes every room offline at once.
`edge gateway`	**Written, not in the path.** It exists for the real constraint — head-ends are LAN-resident behind a hotel firewall and the WAN drops routinely — but actuation currently goes direct from cloud to head-end.
`read-back`	**Impossible with this data.** Not one of the 14,312 commands reads a device. Until vendor read endpoints are captured, confirmations cannot be verified and the interface stays deliberately hedged.
`MCP`	**Semantics only.** The tool model, tiers and scope stripping are implemented; the MCP wire protocol is not.

> **Two data-quality findings worth carrying into planning.** 288 commands across 22 rooms point at a health-check endpoint — they answer 200 and control nothing. And 457 of 477 controllable rooms have at least one capability that two physical devices both answer to, because the zone model has no way to distinguish a bedside lamp from a ceiling light.